

Normalization, Primary & Foreign Keys, Relationships, Join

- Md. Morshedul Arefin

Important Terms

Primary Key (PK):

A column (or set of columns) that uniquely identifies each row in a table. It cannot contain NULL values or duplicates.

Foreign Key (FK):

A column in one table that points to the Primary Key of another table. It enforces referential integrity between child and parent records.

Normalization:

The process of organizing table structures to reduce data redundancy (duplicate info) and improve data integrity.

Unnormalized Table (BAD DESIGN)

Table: students

student_id	student_name	student_email	course_name
1	Habib	habib@example.com	PHP
2	Robin	robin@example.com	Javascript
3	Sabbir	sabbir@example.com	PHP
4	Robin	robin@example.com	PHP

Problems:

- If Robin changes his email, we must update multiple rows.
- If course name is changed from PHP to PHP/MySQL, we must update multiple rows.

Normalized Database Structure (GOOD DESIGN)

Table: students

student_id	student_name	student_email
1	Habib	habib@example.com
2	Robin	robin@example.com
3	Sabbir	sabbir@example.com

Table: courses

course_id	course_name
1	PHP
2	Javascript

Normalized Database Structure (GOOD DESIGN)

Table: enrollments

id	student_id	course_id
1	1	1
2	2	2
3	3	1
4	4	1

One-to-One (1:1) Relationship

A One-to-One (1:1) Relationship exists when a single record in Table A is associated with exactly one record in Table B, and vice versa.

Table: students

id	name	email	age
1	Peter	peter@example.com	20
2	Smith	smith@example.com	22

Make student_id UNIQUE and Foreign Key

Table: student_profiles

id	student_id	nid
1	1	12345
2	2	12789

Steps in phpMyAdmin

Step 1:

Make Storage engine of each table as: InnoDB

Go to table > operations tab > select storage engine

Steps in phpMyAdmin

← Server: localhost:8889 » Database: class_12 » Table: students

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#) [Privileges](#) [Operations](#)

Move table to (database.table)

class_12 . students

☐ Add AUTO_INCREMENT value

☒ Adjust privileges ⓘ

Table options

Rename table to students ☒ Adjust privileges ⓘ

Table comments

Storage engine ⓘ InnoDB

Steps in phpMyAdmin

← Server: localhost:8889 » Database: class_12 » Table: student_profiles

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#) [Privileges](#) [Operations](#)

Move table to (database.table)

class_12 . student_profiles

☐ Add AUTO_INCREMENT value

☒ Adjust privileges ⓘ

Table options

Rename table to student_profiles ☒ Adjust privileges ⓘ

Table comments

Storage engine ⓘ InnoDB

Steps in phpMyAdmin

Step 2:

Primary key and Foreign key data types must be same.

INT(11) or BIGINT and UNSIGNED = Make sure those are same

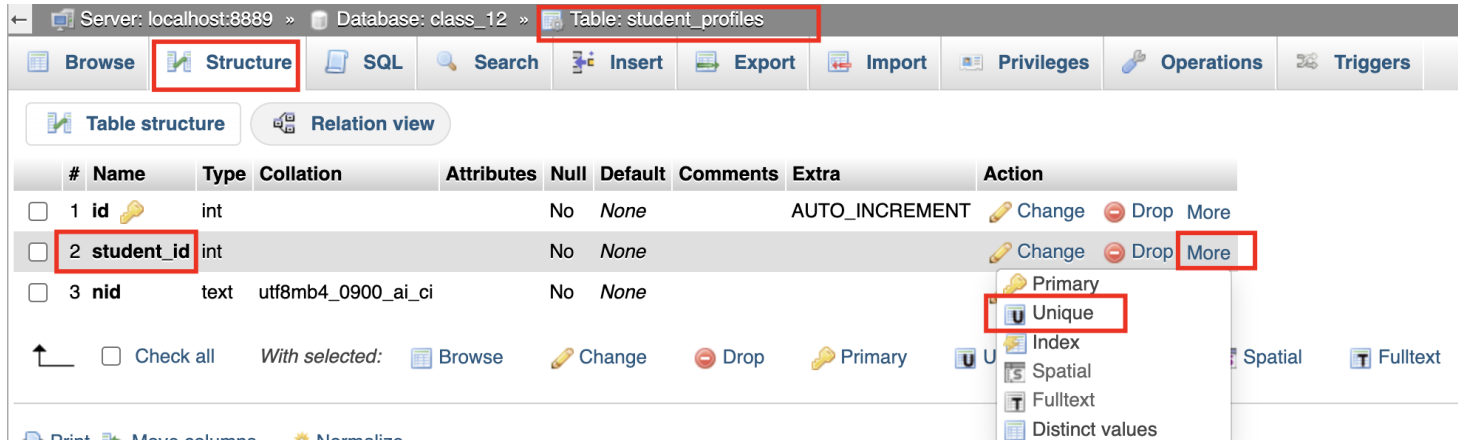
Steps in phpMyAdmin

Step 3:

Make student_id as unique.

Go to student_profiles table and then structure tab.

In the same row of student_id, go rightmost side and click on more and then select unique. From popup, click Ok.



The screenshot shows the phpMyAdmin interface for the 'student_profiles' table. The 'Structure' tab is active. The table structure is as follows:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int			No	None		AUTO_INCREMENT	Change Drop More
2	student_id	int			No	None			Change Drop More
3	nid	text	utf8mb4_0900_ai_ci		No	None			

The 'More' button for the 'student_id' column is clicked, opening a dropdown menu with the following options:

- Primary
- Unique
- Index
- Spatial
- Fulltext
- Distinct values

Steps in phpMyAdmin

Step 4:

Now go to Relation View.

← Server: localhost:8889 » Database: class_12 » Table: student_promises

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#) [Privileges](#) [Operations](#) [Triggers](#)

[Table structure](#) [Relation view](#)

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 id	int			No	None		AUTO_INCREMENT	Change Drop More
☐	2 student_id	int			No	None			Change Drop More
☐	3 nid	text	utf8mb4_0900_ai_ci		No	None			Change Drop More

↑ ☐ Check all With selected: [Browse](#) [Change](#) [Drop](#) [Primary](#) [Unique](#) [Index](#) [Spatial](#) [Fulltext](#)

Steps in phpMyAdmin

Step 5:

You will see this:

Server: localhost:8889 » Database: class_12 » Table: student_profiles

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#) [Privileges](#) [Operations](#) [Triggers](#)

[Table structure](#) [Relation view](#)

Foreign key constraints

Actions	Constraint properties	Column	Foreign key constraint (INNODB)		
			Database	Table	Column
	Constraint name		class_12		
	ON DELETE RESTRICT	ON UPDATE RESTRICT	+ Add column		

[+ Add constraint](#)

[Preview SQL](#) [Save](#)

Steps in phpMyAdmin

Constraint Name: fk_student_profile (give any name like this)

Column: student_id

Database: class_12

Table: students

Column: id

ON DELETE: CASCADE = It is because, if main student is deleted, his profile will also be deleted.

ON UPDATE: CASCADE

Steps in phpMyAdmin

Difference between CASCADE and RESTRICT

CASCADE = When you will delete a student from students table, that student will also be deleted from student_profiles table

RESTRICT = When you will try to delete a student from students table, phpMyAdmin will not allow you to delete this and show error, until you delete that entry from student_profiles table

One-to-Many (1:N) Relationship

- A One-to-Many (1:N) Relationship is the most common relationship in relational databases.
- It occurs when a single record in Table A can link to multiple records in Table B, but each record in Table B links back to only one record in Table A.

Steps are like One-to-One, except making the column UNIQUE.

In a 1-to-Many (1:N) relationship, one parent record can be linked to multiple child records. Therefore, the Foreign Key column must be allowed to store duplicate values.

Many-to-Many (M:N) Relationship

- A Many-to-Many (M:N) Relationship occurs when multiple records in Table A relate to multiple records in Table B.
- Relational databases cannot directly link two tables in an M:N structure.
- To solve this, we create a third table called a Junction Table (or Pivot Table).
- Pivot Table contains foreign keys referencing both primary tables.

Steps in phpMyAdmin

Table: students

id	name
1	Peter
2	Smith

Table: courses

id	name
1	PHP
2	Javascript

Table: course_students

id
course_id
student_id

A student can have many courses.

A course can have many students.

Steps in phpMyAdmin

Add a composite UNIQUE Key.

Server: localhost:8889 » Database: class_12 » Table: course_students

Browse Structure SQL Search Insert Export Import Privileges Operations Triggers

Table structure Relation view

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	id	int			No	None		AUTO_INCREMENT	Change Drop More
☒	2	course_id	int			No	None			Change Drop More
☒	3	student_id	int			No	None			Change Drop More

Check all With selected: Browse Change Drop Primary Unique Index Spatial Fulltext

Steps in phpMyAdmin

Add Foreign Keys in Relation View

← Server: localhost:8889 » Database: class_12 » Table: course_students

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#) [Privileges](#) [Operations](#)

[Table structure](#) [Relation view](#)

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	id 	int			No	None		AUTO_INCREMENT	 Change  Drop More
☐	2	course_id 	int			No	None			 Change  Drop More
☐	3	student_id 	int			No	None			 Change  Drop More

Steps in phpMyAdmin

Actions	Constraint properties	Column 	Foreign key constraint (INNODB)		
			Database	Table	Column
	fk_student_id ON DELETE CASCADE ON UPDATE CASCADE	student_id  + Add column	class_12 	students 	id 
	fk_course_id ON DELETE CASCADE ON UPDATE CASCADE	course_id  + Add column	class_12 	courses 	id 
+ Add constraint					

JOIN Types

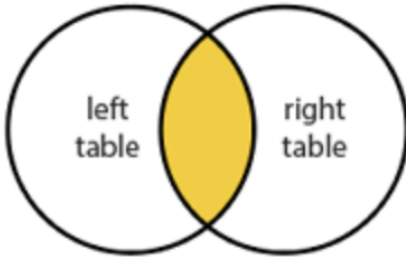
A JOIN clause is used to combine rows from two or more tables based on a related column between them (typically linking a Foreign Key in one table to a Primary Key in another).

Main Types of JOINS:

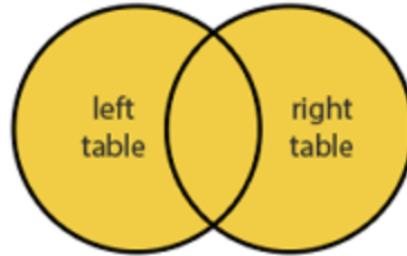
1. INNER JOIN
2. LEFT JOIN / LEFT OUTER JOIN
3. RIGHT JOIN / RIGHT OUTER JOIN
4. FULL OUTER JOIN

JOIN Types

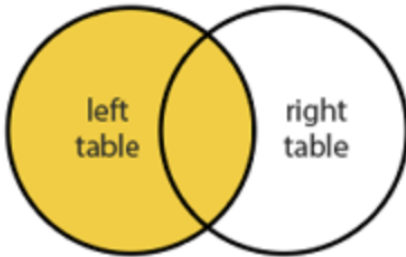
JOIN



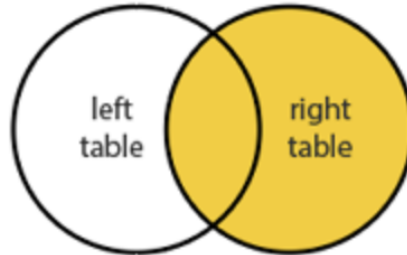
FULL JOIN



LEFT JOIN



RIGHT JOIN



INNER JOIN

- Returns only the records that have matching values in both tables.
- An INNER JOIN matches rows from two tables based on a common column (like a Foreign Key). If a row in the first table does not have a corresponding match in the second table, it will not appear in the result set.

Table: departments

id	name
1	CSE
2	EEE
3	MATH

Table: students

id	department_id	name	mobile
1	1	Sabbir	11111
2	2	Robin	22222
3	<i>NULL</i>	Rahim	33333

INNER JOIN

```
SELECT
    s.id,
    s.name as student_name,
    s.mobile,
    d.name as department_name
FROM students s
INNER JOIN departments d
ON s.department_id = d.id;
```

id	student_name	mobile	department_name
1	Sabbir	11111	CSE
2	Robin	22222	EEE

LEFT JOIN

- A LEFT JOIN returns all rows from the left table (students), along with the matching rows from the right table (departments).
- If there is no match in the right table, MySQL returns NULL for the right table's columns.

Table: departments

id	name
1	CSE
2	EEE
3	MATH

Table: students

id	department_id	name	mobile
1	1	Sabbir	11111
2	2	Robin	22222
3	<i>NULL</i>	Rahim	33333

LEFT JOIN

```
SELECT
    s.id,
    s.name as student_name,
    s.mobile,
    d.name as department_name
FROM students s
LEFT JOIN departments d
ON s.department_id = d.id;
```

id	student_name	mobile	department_name
1	Sabbir	11111	CSE
2	Robin	22222	EEE
3	Rahim	33333	<i>NULL</i>

RIGHT JOIN

- A RIGHT JOIN returns all rows from the right table (departments), along with the matching rows from the left table (students).
- If there is no matching record in the left table, MySQL returns NULL for the left table's columns.

Table: departments

id	name
1	CSE
2	EEE
3	MATH

Table: students

id	department_id	name	mobile
1	1	Sabbir	11111
2	2	Robin	22222
3	NULL	Rahim	33333

RIGHT JOIN

```
SELECT  
    s.id,  
    s.name as student_name,  
    s.mobile,  
    d.name as department_name  
FROM students s  
RIGHT JOIN departments d  
ON s.department_id = d.id;
```

id	student_name	mobile	department_name
1	Sabbir	11111	CSE
2	Robin	22222	EEE
NULL	NULL	NULL	MATH

FULL JOIN

- A FULL OUTER JOIN returns all records when there is a match in either the left table or the right table.
- MySQL does NOT natively support the FULL OUTER JOIN syntax. Instead, we achieve a FULL JOIN in MySQL by combining a LEFT JOIN and a RIGHT JOIN using the UNION operator.

Table: departments

id	name
1	CSE
2	EEE
3	MATH

Table: students

id	department_id	name	mobile
1	1	Sabbir	11111
2	2	Robin	22222
3	<i>NULL</i>	Rahim	33333

FULL JOIN

SELECT

s.id, s.name as student_name,
s.mobile, d.name as department_name

FROM students s

LEFT JOIN departments d

ON s.department_id = d.id

UNION

SELECT

s.id, s.name as student_name,
s.mobile, d.name as department_name

FROM students s

RIGHT JOIN departments d

ON s.department_id = d.id;

id	student_name	mobile	department_name
1	Sabbir	11111	CSE
2	Robin	22222	EEE
3	Rahim	33333	NULL
NULL	NULL	NULL	MATH